

# Analyzing photodetectors via lock-in amplifier integrations

**Shashwata Ghosh (2022B5A31036H)**

PHY F366 – Laboratory Project

Under the supervision of  
**Debanjan Polley**

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS OF  
PHY F366: LABORATORY PROJECT

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE  
PILANI (RAJASTHAN)  
HYDERABAD CAMPUS  
(March 2026)

# Experimental Setup

## Hardware Components

- Holmarc EM5000 Electromagnet (current-controlled)
- Stanford Research SR860 Lock-in Amplifier
- Basler ace-1920 USB Monochrome Camera
- A PC on which I run the codes via Jupyter Notebook

## Software Stack

- Python
- PyVISA (Instrument Control)
- Matplotlib (plotting)
- Vendor DeviceInterface API (Electromagnet control)
- OpenCV + pypylon (imaging and measurement)

# Lock-in amplifier integration

**Goal:** To read electrical response synchronized with magnetic field.

## Method:

1. Connected SR860 with VISA
2. Verified communication
3. Accessed X, Y, R outputs in Python and calculated average values

## Outcome:

1. Real-time lock-in signal acquisition achieved.
2. Lock-in data accessible during magnetic sweeps.

```
In [8]: from srsinst.sr860 import SR860
import pyvisa
rm = pyvisa.ResourceManager()
rm.list_resources()

Out[8]: ('USB0::0x0506::0x2000::006521::INSTR', 'ASRL1::INSTR', 'ASRL3::INSTR')
```

```
In [9]: lockin = SR860('visa', 'USB0::0x0506::0x2000::006521::INSTR')
lockin.check_id()
print(lockin.get_status())
```

Frequency: 7.988065e+02 Hz  
Phase: -122.796042 deg  
Amplitude: 0.5000 V  
DC level: 0.0000 V  
Harmonic: 1

LIA status bit 0, CH10V is set, LIA status bit 1, CH20V is set, LIA status bit 3, UNLK is set, LIA status bit 4, RANGE is set, LIA status bit 8, DAT10V is set, LIA status bit 9, DAT20V is set, LIA status bit 10, DAT30V is set, Event status bit 6, URQ is set,

```
lockin = SR860('visa', 'USB0::...')
lockin_x = lockin.data.value['X']
```

# Electromagnet control via Python

**Goal:** Programmatically set magnet current

**Method:**

1. Serial connection via vendor API
2. Constant current mode
3. Field readback sensor

**Outcome:**

1. Reliable current control from  $-3.5$  A to  $+3.5$  A
2. Magnetic field readback available

```
DeviceInterface.config_serialport("COM3")
DeviceInterface.connect()
DeviceInterface.electromagnet_on()
DeviceInterface.set_field("Constant Current", I)
```

```
start_current = -3.5
stop_current = 3.5
step = 0.5

currents = np.arange(start_current, stop_current + step, step)

for current in currents:
    DeviceInterface.set_field(Mode, current)
```

# Magnetic field vs Current Sweep

**Goal:** Characterise electromagnet calibration

**Method:**

- Sweep current from -3.5 A to +3.5 A
- Wait stabilization time
- Read magnetic field sensor

**Outcome:**

- Linear B vs I relationship obtained
- Calibration curves generated.

```
for current in currents:
    DeviceInterface.set_field(Mode, current)
    time.sleep(settle_time)
    DeviceInterface.read_magnetic_field()
    B = deviceControl.x_moved
```

# Live plotting of field vs current

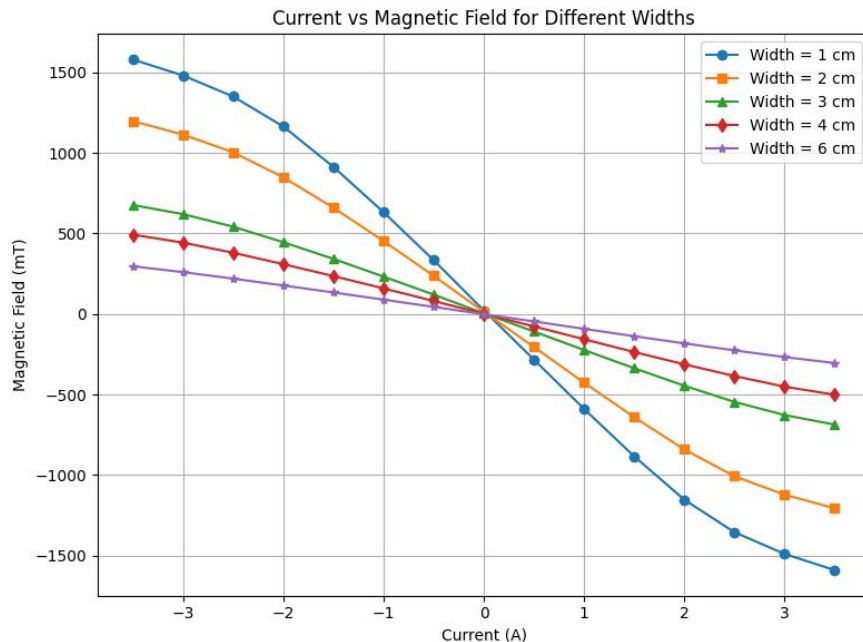
**Goal:** Visualise sweep in real time

**Method:**

- Incremental data storage
- Live matplotlib update in Jupyter NB

**Result:**

- Real time monitoring of magnet response
- Immediate detection of anomalies



# Lock-in X vs current measurement

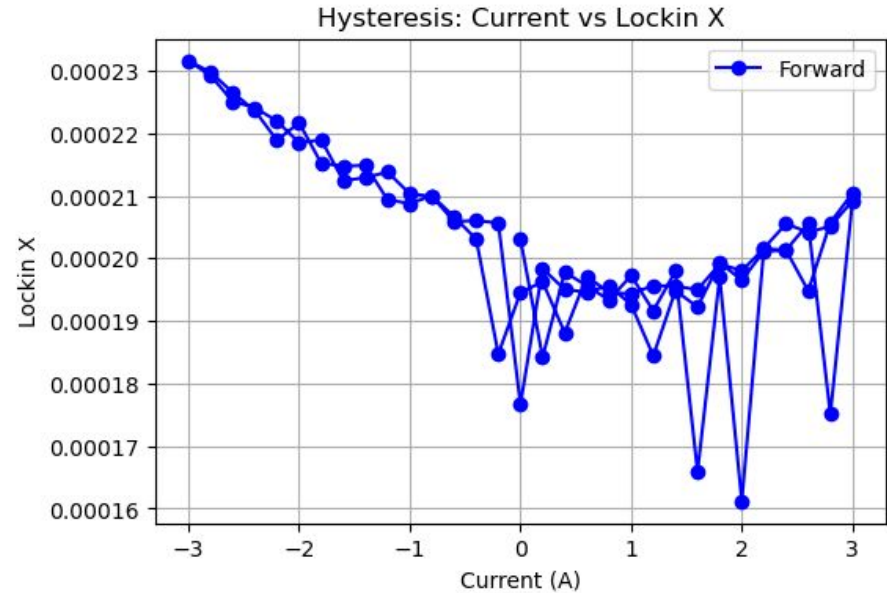
**Goal:** Measure sample electrical response vs magnetic field

**Method:**

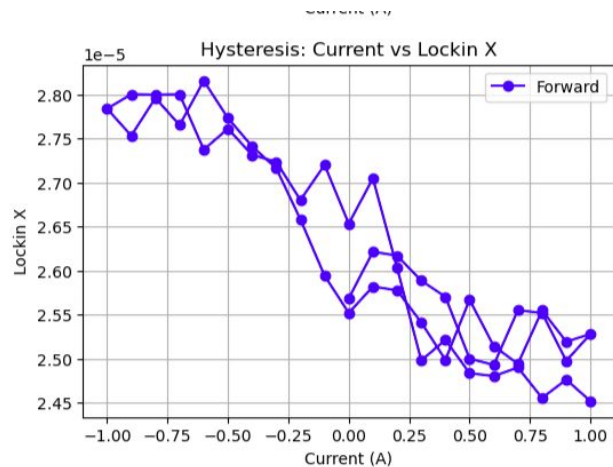
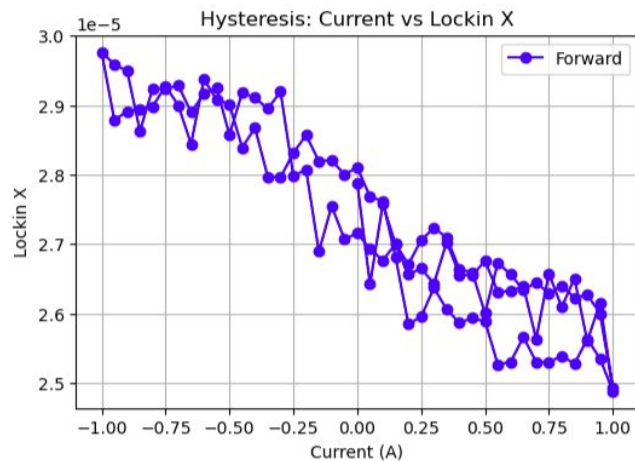
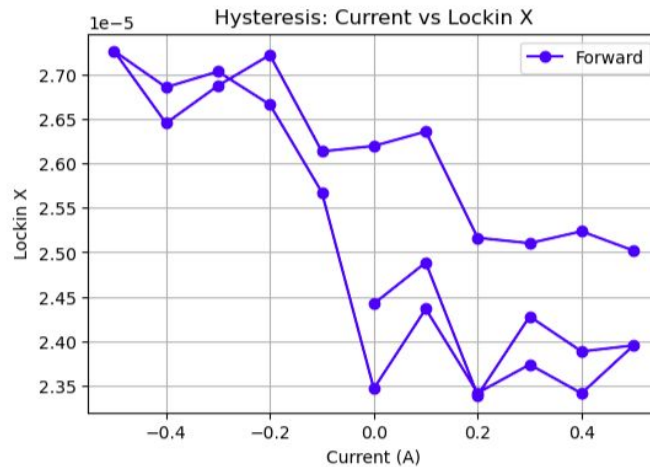
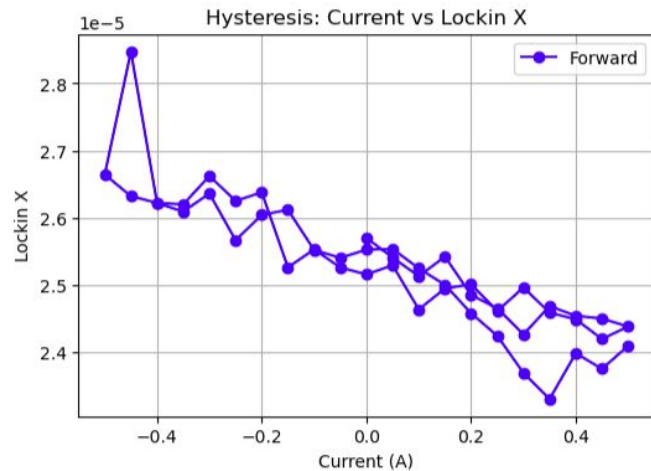
1. Set magnet current
2. Wait stabilisation
3. Read lock-in X channel

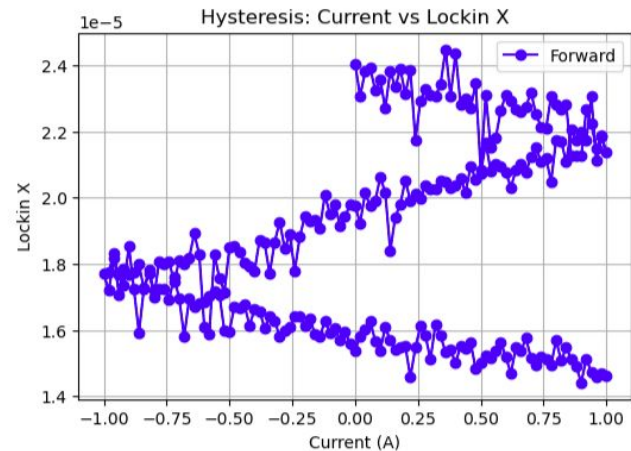
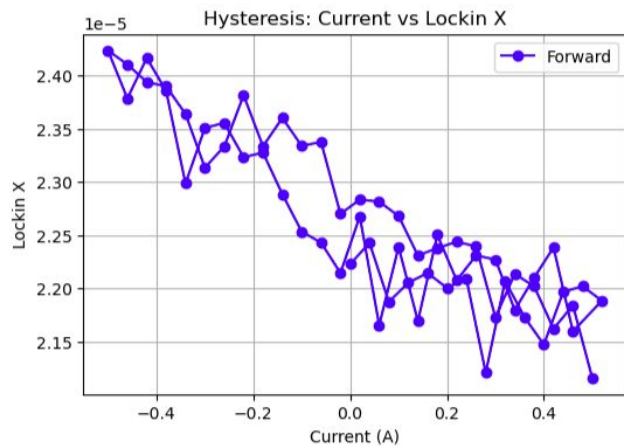
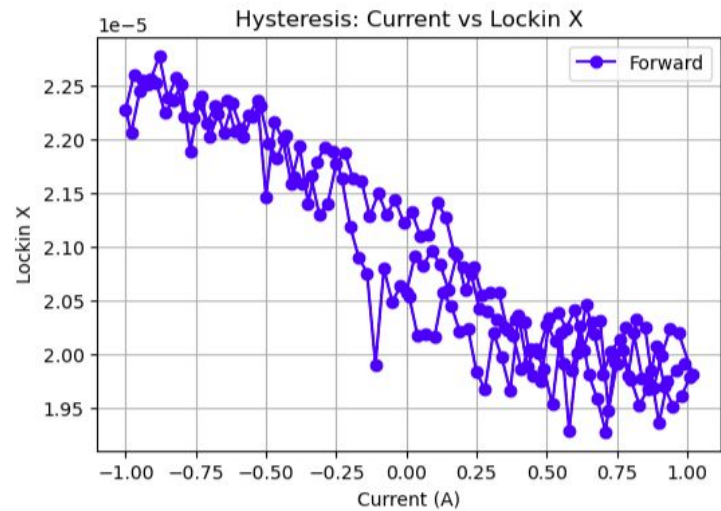
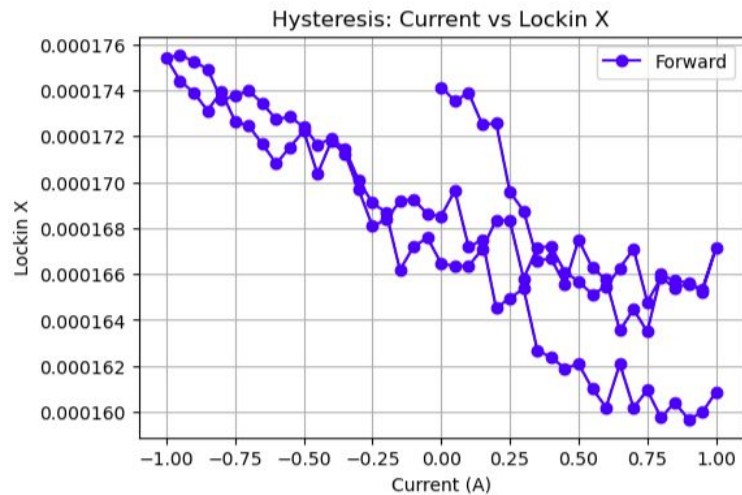
**Outcome:**

1. Electrical response mapped vs magnetic field
2. Data saved for analysis









# Hysteresis Measurement (Bidirectional Sweep)

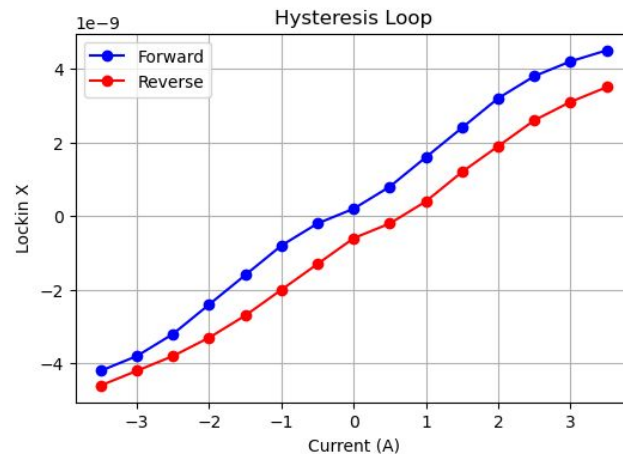
**Goal:** Observe magnetic/electrical hysteresis

**Method:**

- Sweep -3.5 A to + 3.5 A to -3.5 A
- Record lock-in X at each step

**Outcome:**

- Forward and reverse curves obtained
- Hysteresis loop structure obtained



```
step = 0.5

up    = np.arange(0, 3.5 + step, step)
down  = np.arange(3.5, 0 - step, -step)
neg   = np.arange(0, -3.5 - step, -step)
back  = np.arange(-3.5, 0 + step, step)

current = np.concatenate((up, down, neg, back, up))
print(current)
```

```
[ 0.  0.5  1.  1.5  2.  2.5  3.  3.5  3.5  3.  2.5  2.  1.5  1.
 0.5  0.  0. -0.5 -1. -1.5 -2. -2.5 -3. -3.5 -3.5 -3. -2.5 -2.
-1.5 -1. -0.5  0.  0.  0.5  1.  1.5  2.  2.5  3.  3.5]
```

# Basler Camera Integration

Goal: Capture images via Python

Method:

1. Install pypylon SDK
2. Detected USB3 Camera
3. Grabbed frames in Python

Outcome:

1. Successful image acquisition
2. Live video streaming using  
*Continuous mode*

```
from pypylon import pylon
import cv2

# Connect to the first detected camera
camera = pylon.InstantCamera(pylon.TlFactory.GetInstance().CreateFirstDevice())
camera.Open()

# Optional: set monochrome 8-bit format
camera.PixelFormat.SetValue("Mono8")
camera.ExposureTime.SetValue(5000.0) # microseconds

# Grab one image
camera.StartGrabbing(pylon.GrabStrategy_LatestImageOnly)
converter = pylon.ImageFormatConverter()
converter.OutputPixelFormat = pylon.PixelType_Mono8
converter.OutputBitAlignment = pylon.OutputBitAlignment_MsbAligned

grabResult = camera.RetrieveResult(5000, pylon.TimeoutHandling_ThrowException)

if grabResult.GrabSucceeded():
    image = converter.Convert(grabResult)
    img = image.GetArray()
    cv2.imshow("Basler Mono Image", img)
    cv2.waitKey(0)

grabResult.Release()
camera.StopGrabbing()
camera.Close()
cv2.destroyAllWindows()
```

# Distance measurement using OpenCV

**Goal:** Measure real world dimensions in images

**Method:**

- Pixel distance between points
- Calibration: micrometers per pixels (scale)
- Overlay measurement on the image

**Features:**

- Interactive point selection
- Multiple measurements
- On-image annotation

```
def click(event, x, y, flags, param):
    global points, img_display

    if event == cv2.EVENT_LBUTTONDOWN:
        points.append((x, y))
        cv2.circle(img_display, (x, y), 4, (0,255,0), -1)

        # When 2 points selected → measure
        if len(points) == 2:
            p1 = np.array(points[0])
            p2 = np.array(points[1])

            pixel_dist = np.linalg.norm(p1 - p2)
            real_dist = pixel_dist * um_PER_PIXEL

            # Draw line
            cv2.line(img_display, points[0], points[1], (0,255,0), 2)

            # Midpoint for text
            mx = int((points[0][0] + points[1][0]) / 2)
            my = int((points[0][1] + points[1][1]) / 2)

            # Put measurement text on image
            cv2.putText(
                img_display,
                f"{real_dist:.3f} um",
                (mx, my),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.6,
                (0,255,0),
                2
            )

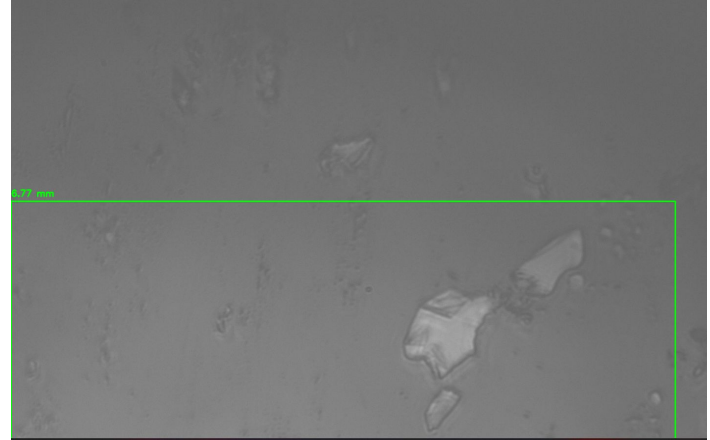
            # Reset for next measurement
            points.clear()

cv2.imshow("Measurement Window", img_display)
```

# Interactive Measurement Tool

## Capabilities:

- Draw unlimited measurement lines
- Distance displayed on top of the line and annotated on the photo
- Press ESC to exit



# Conclusion

Up to the mid-semester stage, I have built a complete experimental setup to control the electromagnet and measure its response using a lock-in amplifier. I wrote Python code to automatically sweep current, record magnetic field and lock-in data, and plot the results live, including hysteresis behavior. I also connected the Basler camera and created an OpenCV tool to capture images and measure real-world dimensions from them. This setup now allows me to perform magnetic measurements and optical analysis in a consistent and repeatable way.

